

## **CARRERA DESARROLLO DE SOFTWARE**

---

**DISEÑO DE SOFTWARE - [M-DSOF-AM-DS33-7547] – [7545]**

### **TAREA PRÁCTICA – CONEXIÓN A DIFERENTES BASES DE DATOS**

**ARIAS FAREZ, JESSICA ALEXANDRA**

**Profesora: FRANCO ROCHA YADIRA GUISSOLA**

[yadira.franco@cenestur.edu.ec](mailto:yadira.franco@cenestur.edu.ec)

**Quito, Ecuador**

**2026**

# CODIGOS DE INTELIJI

## 1. CLEVER CLOUD

```
package com.veterinaria;

import javafx.beans.property.SimpleIntegerProperty;
import javafx.beans.property.SimpleStringProperty;
import javafx.collections.FXCollections;
import javafx.collections.ObservableList;
import javafx.fxml.FXML;
import javafx.scene.control.Alert;
import javafx.scene.control.ComboBox;
import javafx.scene.control.TableColumn;
import javafx.scene.control.TableView;
import javafx.scene.control.TextField;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Statement;

public class VeterinariaController {

    @FXML
    private TextField txtPropietario, txtMascota, txtEdad,
    txtDiagnostico;
    @FXML
    private ComboBox<String> cmbEspecie;
    @FXML
    private TableView<Mascota> tablaMascotas;
    @FXML
    private TableColumn<Mascota, Integer> colId, colEdad;
    @FXML
    private TableColumn<Mascota, String> colPropietario, colMascota,
    colEspecie, colDiagnostico;

    private final String URL = "jdbc:mysql://b14lcvoqgdj2ovk74kua-
mysql.services.clever-cloud.com:3306/b14lcvoqgdj2ovk74kua";
    private final String USUARIO = "uwvtckz0kzes3las";
    private final String CLAVE = "H8miDlYXblwWtgFXggwo";

    private final ObservableList<Mascota> lista =
    FXCollections.observableArrayList();

    @FXML
    public void initialize() {
        cmbEspecie.getItems().addAll("Perro", "Gato", "Ave",
        "Conejo", "Otro");

        colId.setCellValueFactory(d -> new
        SimpleIntegerProperty(d.getValue().id).asObject());
        colPropietario.setCellValueFactory(d -> new
        SimpleStringProperty(d.getValue().propietario));
        colMascota.setCellValueFactory(d -> new
        SimpleStringProperty(d.getValue().mascota));
```

```

        colEspecie.setCellValueFactory(d -> new
SimpleStringProperty(d.getValue().especie));
        colEdad.setCellValueFactory(d -> new
SimpleIntegerProperty(d.getValue().edad).asObject());
        colDiagnostico.setCellValueFactory(d -> new
SimpleStringProperty(d.getValue().diagnostico));

        tablaMascotas.setItems(lista);
        cargarDatos();

        tablaMascotas.setOnMouseClicked(e -> {
            Mascota m =
tablaMascotas.getSelectionModel().getSelectedItem();
            if (m != null) {
                txtPropietario.setText(m.propietario);
                txtMascota.setText(m.mascota);
                cmbEspecie.setValue(m.especie);
                txtEdad.setText(String.valueOf(m.edad));
                txtDiagnostico.setText(m.diagnostico);
            }
        });
    }

    Connection conectar() throws SQLException {
        return DriverManager.getConnection(URL, USUARIO, CLAVE);
    }

    void cargarDatos() {
        lista.clear();
        String sql = "SELECT * FROM mascotas ORDER BY id DESC";

        try (Connection c = conectar();
            Statement s = c.createStatement();
            ResultSet r = s.executeQuery(sql)) {

            while (r.next()) {
                lista.add(new Mascota(
                    r.getInt("id"),
                    r.getString("propietario"),
                    r.getString("mascota"),
                    r.getString("especie"),
                    r.getInt("edad"),
                    r.getString("diagnostico")
                ));
            }
        } catch (Exception e) {
            mensaje("Error", e.getMessage());
        }
    }

    @FXML
    void guardar() {
        if (!validar()) return;

        String sql = "INSERT INTO mascotas(propietario, mascota,
especie, edad, diagnostico) VALUES(?, ?, ?, ?, ?)";

        try (Connection c = conectar();
            PreparedStatement p = c.prepareStatement(sql)) {

            p.setString(1, txtPropietario.getText());

```

```

        p.setString(2, txtMascota.getText());
        p.setString(3, cmbEspecie.getValue());
        p.setInt(4, Integer.parseInt(txtEdad.getText()));
        p.setString(5, txtDiagnostico.getText());

        p.executeUpdate();
        limpiar();
        cargarDatos();
        mensaje("Correcto", "Mascota registrada correctamente.");
    } catch (Exception e) {
        mensaje("Error", e.getMessage());
    }
}

@FXML
void actualizar() {
    Mascota m =
tablaMascotas.getSelectionModel().getSelectedItem();
    if (m == null) {
        mensaje("Aviso", "Seleccione un registro.");
        return;
    }

    if (!validar()) return;

    String sql = "UPDATE mascotas SET propietario=?, mascota=?,
especie=?, edad=?, diagnostico=? WHERE id=?";

    try (Connection c = conectar();
        PreparedStatement p = c.prepareStatement(sql)) {

        p.setString(1, txtPropietario.getText());
        p.setString(2, txtMascota.getText());
        p.setString(3, cmbEspecie.getValue());
        p.setInt(4, Integer.parseInt(txtEdad.getText()));
        p.setString(5, txtDiagnostico.getText());
        p.setInt(6, m.id);

        p.executeUpdate();
        limpiar();
        cargarDatos();
    } catch (Exception e) {
        mensaje("Error", e.getMessage());
    }
}

@FXML
void eliminar() {
    Mascota m =
tablaMascotas.getSelectionModel().getSelectedItem();
    if (m == null) {
        mensaje("Aviso", "Seleccione una mascota.");
        return;
    }

    String sql = "DELETE FROM mascotas WHERE id=?";

    try (Connection c = conectar();
        PreparedStatement p = c.prepareStatement(sql)) {

        p.setInt(1, m.id);

```

```

        p.executeUpdate();
        limpiar();
        cargarDatos();
    } catch (Exception e) {
        mensaje("Error", e.getMessage());
    }
}

@FXML
void nuevo() {
    limpiar();
    tablaMascotas.getSelectionModel().clearSelection();
}

boolean validar() {
    if (txtPropietario.getText().isBlank() ||
txtMascota.getText().isBlank() || cmbEspecie.getValue() == null) {
        mensaje("Validación", "Complete propietario, mascota y
especie.");
        return false;
    }

    try {
        if (Integer.parseInt(txtEdad.getText()) < 0) {
            throw new Exception();
        }
    } catch (Exception e) {
        mensaje("Validación", "La edad debe ser un número
válido.");
        return false;
    }

    return true;
}

void limpiar() {
    txtPropietario.clear();
    txtMascota.clear();
    cmbEspecie.setValue(null);
    txtEdad.clear();
    txtDiagnostico.clear();
}

void mensaje(String t, String m) {
    Alert a = new Alert(Alert.AlertType.INFORMATION, m);
    a.setTitle(t);
    a.setHeaderText(null);
    a.showAndWait();
}

public static class Mascota {
    int id, edad;
    String propietario, mascota, especie, diagnostico;

    Mascota(int i, String p, String m, String e, int ed, String
d) {
        this.id = i;
        this.propietario = p;
        this.mascota = m;
        this.especie = e;
        this.edad = ed;
    }
}

```

```
        this.diagnostico = d;
    }
}
}
```

## 2. MONGODB COMPAS

```
package com.veterinaria;
```

```
import com.mongodb.client.MongoClient;
```

```
import com.mongodb.client.MongoClients;
```

```
import com.mongodb.client.MongoCollection;
```

```
import com.mongodb.client.MongoDatabase;
```

```
import javafx.beans.property.*;
```

```
import javafx.collections.*;
```

```
import javafx.fxml.FXML;
```

```
import javafx.scene.control.*;
```

```
import org.bson.Document;
```

```
import org.bson.types.ObjectId;
```

```
import static com.mongodb.client.model.Filters.eq;
```

```
import static com.mongodb.client.model.Updates.*;
```

```
public class VeterinariaController {
```

@FXML

TextField **txtPropietario, txtMascota, txtEdad, txtDiagnostico;**

@FXML

ComboBox<String> **cmbEspecie;**

@FXML

TableView<Mascota> **tablaMascotas;**

@FXML

TableColumn<Mascota, String> **colId;**

@FXML

TableColumn<Mascota, Integer> **colEdad;**

@FXML

TableColumn<Mascota, String> **colPropietario,**

**colMascota,**

**colEspecie,**

**colDiagnostico;**

*// CONEXIÓN A MONGODB*

**private final** String **URI = "mongodb://localhost:27017";**

**private** MongoClient **mongoClient;**

**private** MongoDBDatabase **database;**

```
private MongoCollection<Document> coleccion;
```

```
private final ObservableList<Mascota> lista =
```

```
    FXCollections.observableArrayList();
```

```
@FXML
```

```
public void initialize() {
```

```
    // Conectar con MongoDB
```

```
    conectar();
```

```
    cmbEspecie.getItems().addAll(
```

```
        "Perro",
```

```
        "Gato",
```

```
        "Ave",
```

```
        "Conejo",
```

```
        "Otro"
```

```
    );
```

```
    colId.setCellValueFactory(
```

```
        d -> new SimpleStringProperty(d.getValue().id)
```

```
    );
```

```
    colPropietario.setCellValueFactory(
```

```
        d -> new SimpleStringProperty(d.getValue().propietario)
```

```
    );
```



```
colMascota.setCellValueFactory(  
    d -> new SimpleStringProperty(d.getValue()).mascota)  
);
```

```
colEspecie.setCellValueFactory(  
    d -> new SimpleStringProperty(d.getValue()).especie)  
);
```

```
colEdad.setCellValueFactory(  
    d -> new SimpleIntegerProperty(d.getValue()).edad).asObject()  
);
```

```
colDiagnostico.setCellValueFactory(  
    d -> new SimpleStringProperty(d.getValue()).diagnostico)  
);
```

```
tablaMascotas.setItems(lista);
```

```
cargarDatos();
```

```
tablaMascotas.setOnMouseClicked(e -> {
```

```
    Mascota m =
```

```
        tablaMascotas.getSelectionModel().getSelectedItem();
```

```
if (m != null) {

    txtPropietario.setText(m.propietario);

    txtMascota.setText(m.mascota);

    cmbEspecie.setValue(m.especie);

    txtEdad.setText(
        String.valueOf(m.edad)
    );

    txtDiagnostico.setText(
        m.diagnostico
    );
}

});

}
```

```
// =====

// CONEXIÓN

// =====
```

```
void conectar() {
```

```
    try {
```

```
mongoClient =
```

```
    MongoClients.create(URI);
```

```
database =
```

```
    mongoClient.getDatabase("veterinaria");
```

```
coleccion =
```

```
    database.getCollection("mascotas");
```

```
System.out.println(
```

```
    "Conectado correctamente a MongoDB"
```

```
);
```

```
} catch (Exception e) {
```

```
    mensaje(
```

```
        "Error de conexión",
```

```
        e.getMessage()
```

```
    );
```

```
}
```

```
}
```

```
// =====
```

```
// MOSTRAR DATOS
```

```
// =====
```

```
void cargarDatos() {

    lista.clear();

    try {

        for (Document doc : coleccion.find()) {

            String id =

                doc.getObjectId("_id").toHexString();

            String propietario =

                doc.getString("propietario");

            String mascota =

                doc.getString("mascota");

            String especie =

                doc.getString("especie");

            Integer edad =

                doc.getInteger("edad");

            String diagnostico =

                doc.getString("diagnostico");
```

```
        lista.add(  
            new Mascota(  
                id,  
                propietario,  
                mascota,  
                especie,  
                edad,  
                diagnostico  
            )  
        );  
    }  
  
    } catch (Exception e) {  
  
        mensaje(  
            "Error",  
            e.getMessage()  
        );  
    }  
}  
  
// =====  
  
// INSERTAR  
  
// =====
```

@FXML

**void** guardar() {

if (!validar())

**return;**

**try** {

Document documento =

**new** Document(

**"propietario",**

**txtPropietario.getText()**

)

.append(

**"mascota",**

**txtMascota.getText()**

)

.append(

**"especie",**

**cmbEspecie.getValue()**

)

.append(

**"edad",**

*Integer.parseInt*(

**txtEdad.getText()**

)

```
        )

        .append(

            "diagnostico",

            txtDiagnostico.getText()

        );

coleccion.insertOne(documento);

limpiar();

cargarDatos();

mensaje(

    "Correcto",

    "Mascota registrada correctamente."

);

} catch (Exception e) {

    mensaje(

        "Error",

        e.getMessage()

    );

}

}
```

```
// =====
```

```
// ACTUALIZAR
```

```
// =====
```

```
@FXML
```

```
void actualizar() {
```

```
    Mascota m =
```

```
        tablaMascotas
```

```
            .getSelectionModel()
```

```
            .getSelectedItem();
```

```
    if (m == null) {
```

```
        mensaje(
```

```
            "Aviso",
```

```
            "Seleccione un registro."
```

```
        );
```

```
        return;
```

```
    }
```

```
    if (!validar())
```

```
        return;
```

```
    try {
```



ObjectId id =

**new** ObjectId(m.**id**);

**coleccion.updateOne**(

*eq*("\_id", id),

*combine*(

*set*(

**"propietario"**,

**txtPropietario.getText()**

),

*set*(

**"mascota"**,

**txtMascota.getText()**

),

*set*(

**"especie"**,

**cmbEspecie.getValue()**

),

*set*(

```
        "edad",

        Integer.parseInt(

            txtEdad.getText()

        )

    ),

    set(

        "diagnostico",

        txtDiagnostico.getText()

    )

)

);

limpiar();

cargarDatos();

mensaje(

    "Correcto",

    "Registro actualizado."

);

} catch (Exception e) {

    mensaje(

        "Error",
```

```

        e.getMessage()

    );

}

}

// =====

// ELIMINAR

// =====

@FXML

void eliminar() {

    Mascota m =

        tablaMascotas

            .getSelectionModel()

            .getSelectedItem();

    if (m == null) {

        mensaje(

            "Aviso",

            "Seleccione una mascota."

        );

        return;

    }

```

```
try {

    ObjectId id =

        new ObjectId(m.id);

    coleccion.deleteOne(

        eq("_id", id)

    );

    limpiar();

    cargarDatos();

    mensaje(

        "Correcto",

        "Mascota eliminada."

    );

} catch (Exception e) {

    mensaje(

        "Error",

        e.getMessage()

    );

}
```

```
}
```

```
// =====
```

```
// NUEVO
```

```
// =====
```

```
@FXML
```

```
void nuevo() {
```

```
    limpiar();
```

```
    tablaMascotas
```

```
        .getSelectionModel()
```

```
        .clearSelection();
```

```
}
```

```
// =====
```

```
// VALIDACIONES
```

```
// =====
```

```
boolean validar() {
```

```
    if (
```

```
        txtPropietario.getText().isBlank()
```

```
        ||
```

```
        txtMascota.getText().isBlank()
```

```
        ||

        cmbEspecie.getValue() == null
    ){

        mensaje(

            "Validación",

            "Complete propietario, mascota y especie."

        );

        return false;
    }
```

```
try {

    int edad =

        Integer.parseInt(

            txtEdad.getText()

        );

    if (edad < 0)

        throw new Exception();

} catch (Exception e) {
```

```
    mensaje(

        "Validación",
```

**"La edad debe ser un número válido."**

);

**return false;**

}

**return true;**

}

// =====

// LIMPIAR

// =====

**void** limpiar() {

**txtPropietario.clear();**

**txtMascota.clear();**

**cmbEspecie.setValue(null);**

**txtEdad.clear();**

**txtDiagnostico.clear();**

}

```
// =====
```

```
// MENSAJE
```

```
// =====
```

```
void mensaje(  
    String titulo,  
    String mensaje  
) {
```

```
    Alert alerta =
```

```
        new Alert(  
            Alert.AlertType.INFORMATION,  
            mensaje  
        );
```

```
    alerta.setTitle(titulo);
```

```
    alerta.setHeaderText(null);
```

```
    alerta.showAndWait();
```

```
}
```

```
// =====
```

```
// CLASE MASCOTA
```

```
// =====
```



```
public static class Mascota {  
  
    String id;  
  
    int edad;  
  
    String propietario,  
        mascota,  
        especie,  
        diagnostico;  
  
    Mascota(  
        String id,  
        String propietario,  
        String mascota,  
        String especie,  
        int edad,  
        String diagnostico  
    ){  
  
        this.id = id;  
  
        this.propietario = propietario;  
  
        this.mascota = mascota;
```

```
    this.especie = especie;
```

```
    this.edad = edad;
```

```
    this.diagnostico = diagnostico;
```

```
    }
```

```
    }
```

```
}
```

### 3. SQL SERVER

```
package com.veterinaria;

import javafx.beans.property.SimpleIntegerProperty;
import javafx.beans.property.SimpleStringProperty;
import javafx.collections.FXCollections;
import javafx.collections.ObservableList;
import javafx.fxml.FXML;
import javafx.scene.control.Alert;
import javafx.scene.control.ComboBox;
import javafx.scene.control.TableColumn;
import javafx.scene.control.TableView;
import javafx.scene.control.TextField;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Statement;

public class VeterinariaController {

    @FXML
    private TextField txtPropietario, txtMascota, txtEdad, txtDiagnostico;
    @FXML
    private ComboBox<String> cmbEspecie;
    @FXML
    private TableView<Mascota> tablaMascotas;
    @FXML
    private TableColumn<Mascota, Integer> colId, colEdad;
    @FXML
    private TableColumn<Mascota, String> colPropietario, colMascota,
colEspecie, colDiagnostico;

    private final String URL =
"jdbc:sqlserver://localhost:1433;databaseName=veterinaria;encrypt=true;trustServerCertificate=true";
```

```

private final String USUARIO = "sa";
private final String CLAVE = "Jessi1995...";

private final ObservableList<Mascota> lista =
FXCollections.observableArrayList();

@FXML
public void initialize() {
    cmbEspecie.getItems().addAll("Perro", "Gato", "Ave", "Conejo",
    "Otro");

    colId.setCellValueFactory(d -> new
SimpleIntegerProperty(d.getValue().id).asObject());
    colPropietario.setCellValueFactory(d -> new
SimpleStringProperty(d.getValue().propietario));
    colMascota.setCellValueFactory(d -> new
SimpleStringProperty(d.getValue().mascota));
    colEspecie.setCellValueFactory(d -> new
SimpleStringProperty(d.getValue().especie));
    colEdad.setCellValueFactory(d -> new
SimpleIntegerProperty(d.getValue().edad).asObject());
    colDiagnostico.setCellValueFactory(d -> new
SimpleStringProperty(d.getValue().diagnostico));

    tablaMascotas.setItems(lista);
    cargarDatos();

    tablaMascotas.setOnMouseClicked(e -> {
        Mascota m =
tablaMascotas.getSelectionModel().getSelectedItem();
        if (m != null) {
            txtPropietario.setText(m.propietario);
            txtMascota.setText(m.mascota);
            cmbEspecie.setValue(m.especie);
            txtEdad.setText(String.valueOf(m.edad));
            txtDiagnostico.setText(m.diagnostico);
        }
    });
}

Connection conectar() throws SQLException {
    return DriverManager.getConnection(URL, USUARIO, CLAVE);
}

void cargarDatos() {
    lista.clear();
    String sql = "SELECT * FROM mascotas ORDER BY id DESC";

    try (Connection c = conectar();
        Statement s = c.createStatement();
        ResultSet r = s.executeQuery(sql)) {

        while (r.next()) {
            lista.add(new Mascota(
                r.getInt("id"),
                r.getString("propietario"),
                r.getString("mascota"),
                r.getString("especie"),
                r.getInt("edad"),
                r.getString("diagnostico")
            ));
        }
    }
}

```

```

        ));
    }
} catch (Exception e) {
    mensaje("Error", e.getMessage());
}
}

@FXML
void guardar() {
    if (!validar()) return;

    String sql = "INSERT INTO mascotas(propietario, mascota, especie,
edad, diagnostico) VALUES(?, ?, ?, ?, ?)";

    try (Connection c = conectar();
        PreparedStatement p = c.prepareStatement(sql)) {

        p.setString(1, txtPropietario.getText());
        p.setString(2, txtMascota.getText());
        p.setString(3, cmbEspecie.getValue());
        p.setInt(4, Integer.parseInt(txtEdad.getText()));
        p.setString(5, txtDiagnostico.getText());

        p.executeUpdate();
        limpiar();
        cargarDatos();
        mensaje("Correcto", "Mascota registrada correctamente.");
    } catch (Exception e) {
        mensaje("Error", e.getMessage());
    }
}

@FXML
void actualizar() {
    Mascota m = tablaMascotas.getSelectionModel().getSelectedItem();
    if (m == null) {
        mensaje("Aviso", "Seleccione un registro.");
        return;
    }

    if (!validar()) return;

    String sql = "UPDATE mascotas SET propietario=?, mascota=?,
especie=?, edad=?, diagnostico=? WHERE id=?";

    try (Connection c = conectar();
        PreparedStatement p = c.prepareStatement(sql)) {

        p.setString(1, txtPropietario.getText());
        p.setString(2, txtMascota.getText());
        p.setString(3, cmbEspecie.getValue());
        p.setInt(4, Integer.parseInt(txtEdad.getText()));
        p.setString(5, txtDiagnostico.getText());
        p.setInt(6, m.id);

        p.executeUpdate();
        limpiar();
        cargarDatos();
    } catch (Exception e) {
        mensaje("Error", e.getMessage());
    }
}

```

```

    }

    @FXML
    void eliminar() {
        Mascota m = tablaMascotas.getSelectionModel().getSelectedItem();
        if (m == null) {
            mensaje("Aviso", "Seleccione una mascota.");
            return;
        }

        String sql = "DELETE FROM mascotas WHERE id=?";

        try (Connection c = conectar();
            PreparedStatement p = c.prepareStatement(sql)) {

            p.setInt(1, m.id);
            p.executeUpdate();
            limpiar();
            cargarDatos();
        } catch (Exception e) {
            mensaje("Error", e.getMessage());
        }
    }

    @FXML
    void nuevo() {
        limpiar();
        tablaMascotas.getSelectionModel().clearSelection();
    }

    boolean validar() {
        if (txtPropietario.getText().isBlank() ||
            txtMascota.getText().isBlank() || cmbEspecie.getValue() == null) {
            mensaje("Validación", "Complete propietario, mascota y
especie.");
            return false;
        }

        try {
            if (Integer.parseInt(txtEdad.getText()) < 0) {
                throw new Exception();
            }
        } catch (Exception e) {
            mensaje("Validación", "La edad debe ser un número válido.");
            return false;
        }

        return true;
    }

    void limpiar() {
        txtPropietario.clear();
        txtMascota.clear();
        cmbEspecie.setValue(null);
        txtEdad.clear();
        txtDiagnostico.clear();
    }

    void mensaje(String t, String m) {
        Alert a = new Alert(Alert.AlertType.INFORMATION, m);
        a.setTitle(t);
    }

```

```

        a.setHeaderText(null);
        a.showAndWait();
    }

    public static class Mascota {
        int id, edad;
        String propietario, mascota, especie, diagnostico;

        Mascota(int i, String p, String m, String e, int ed, String d) {
            this.id = i;
            this.propietario = p;
            this.mascota = m;
            this.especie = e;
            this.edad = ed;
            this.diagnostico = d;
        }
    }
}

```

#### 4. PGADMIN4

```

package com.veterinaria;

import javafx.beans.property.SimpleIntegerProperty;
import javafx.beans.property.SimpleStringProperty;
import javafx.collections.FXCollections;
import javafx.collections.ObservableList;
import javafx.fxml.FXML;
import javafx.scene.control.Alert;
import javafx.scene.control.ComboBox;
import javafx.scene.control.TableColumn;
import javafx.scene.control.TableView;
import javafx.scene.control.TextField;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Statement;

public class VeterinariaController {

    @FXML
    private TextField txtPropietario, txtMascota, txtEdad, txtDiagnostico;
    @FXML
    private ComboBox<String> cmbEspecie;
    @FXML
    private TableView<Mascota> tablaMascotas;
    @FXML
    private TableColumn<Mascota, Integer> colId, colEdad;
    @FXML
    private TableColumn<Mascota, String> colPropietario, colMascota,
colEspecie, colDiagnostico;

    private final String URL =
"jdbc:postgresql://localhost:5432/veterinaria";
    private final String USUARIO = "postgres"; // Usuario por defecto en

```

```

PostgreSQL
    private final String CLAVE = "Jessi1995...";

    private final ObservableList<Mascota> lista =
FXCollections.observableArrayList();

@FXML
    public void initialize() {
        cmbEspecie.getItems().addAll("Perro", "Gato", "Ave", "Conejo",
"Otro");

        colId.setCellValueFactory(d -> new
SimpleIntegerProperty(d.getValue().id).asObject());
        colPropietario.setCellValueFactory(d -> new
SimpleStringProperty(d.getValue().propietario));
        colMascota.setCellValueFactory(d -> new
SimpleStringProperty(d.getValue().mascota));
        colEspecie.setCellValueFactory(d -> new
SimpleStringProperty(d.getValue().especie));
        colEdad.setCellValueFactory(d -> new
SimpleIntegerProperty(d.getValue().edad).asObject());
        colDiagnostico.setCellValueFactory(d -> new
SimpleStringProperty(d.getValue().diagnostico));

        tablaMascotas.setItems(lista);
        cargarDatos();

        tablaMascotas.setOnMouseClicked(e -> {
            Mascota m =
tablaMascotas.getSelectionModel().getSelectedItem();
            if (m != null) {
                txtPropietario.setText(m.propietario);
                txtMascota.setText(m.mascota);
                cmbEspecie.setValue(m.especie);
                txtEdad.setText(String.valueOf(m.edad));
                txtDiagnostico.setText(m.diagnostico);
            }
        });
    }

    Connection conectar() throws SQLException {
        return DriverManager.getConnection(URL, USUARIO, CLAVE);
    }

    void cargarDatos() {
        lista.clear();
        String sql = "SELECT * FROM mascotas ORDER BY id DESC";

        try (Connection c = conectar();
            Statement s = c.createStatement();
            ResultSet r = s.executeQuery(sql)) {

            while (r.next()) {
                lista.add(new Mascota(
                    r.getInt("id"),
                    r.getString("propietario"),
                    r.getString("mascota"),
                    r.getString("especie"),
                    r.getInt("edad"),

```

```

        r.getString("diagnostico")
    ));
    }
} catch (Exception e) {
    mensaje("Error", e.getMessage());
}
}

@FXML
void guardar() {
    if (!validar()) return;

    String sql = "INSERT INTO mascotas(propietario, mascota, especie,
edad, diagnostico) VALUES(?, ?, ?, ?, ?)";

    try (Connection c = conectar();
        PreparedStatement p = c.prepareStatement(sql)) {

        p.setString(1, txtPropietario.getText());
        p.setString(2, txtMascota.getText());
        p.setString(3, cmbEspecie.getValue());
        p.setInt(4, Integer.parseInt(txtEdad.getText()));
        p.setString(5, txtDiagnostico.getText());

        p.executeUpdate();
        limpiar();
        cargarDatos();
        mensaje("Correcto", "Mascota registrada correctamente.");
    } catch (Exception e) {
        mensaje("Error", e.getMessage());
    }
}

@FXML
void actualizar() {
    Mascota m = tablaMascotas.getSelectionModel().getSelectedItem();
    if (m == null) {
        mensaje("Aviso", "Seleccione un registro.");
        return;
    }

    if (!validar()) return;

    String sql = "UPDATE mascotas SET propietario=?, mascota=?,
especie=?, edad=?, diagnostico=? WHERE id=?";

    try (Connection c = conectar();
        PreparedStatement p = c.prepareStatement(sql)) {

        p.setString(1, txtPropietario.getText());
        p.setString(2, txtMascota.getText());
        p.setString(3, cmbEspecie.getValue());
        p.setInt(4, Integer.parseInt(txtEdad.getText()));
        p.setString(5, txtDiagnostico.getText());
        p.setInt(6, m.id);

        p.executeUpdate();
        limpiar();
        cargarDatos();
    } catch (Exception e) {
        mensaje("Error", e.getMessage());
    }
}

```



```

    }
}

@FXML
void eliminar() {
    Mascota m = tablaMascotas.getSelectionModel().getSelectedItem();
    if (m == null) {
        mensaje("Aviso", "Seleccione una mascota.");
        return;
    }

    String sql = "DELETE FROM mascotas WHERE id=?";

    try (Connection c = conectar();
        PreparedStatement p = c.prepareStatement(sql)) {

        p.setInt(1, m.id);
        p.executeUpdate();
        limpiar();
        cargarDatos();
    } catch (Exception e) {
        mensaje("Error", e.getMessage());
    }
}

@FXML
void nuevo() {
    limpiar();
    tablaMascotas.getSelectionModel().clearSelection();
}

boolean validar() {
    if (txtPropietario.getText().isBlank() ||
txtMascota.getText().isBlank() || cmbEspecie.getValue() == null) {
        mensaje("Validación", "Complete propietario, mascota y
especie.");
        return false;
    }

    try {
        if (Integer.parseInt(txtEdad.getText()) < 0) {
            throw new Exception();
        }
    } catch (Exception e) {
        mensaje("Validación", "La edad debe ser un número válido.");
        return false;
    }

    return true;
}

void limpiar() {
    txtPropietario.clear();
    txtMascota.clear();
    cmbEspecie.setValue(null);
    txtEdad.clear();
    txtDiagnostico.clear();
}

void mensaje(String t, String m) {
    Alert a = new Alert(Alert.AlertType.INFORMATION, m);

```

```

        a.setTitle(t);
        a.setHeaderText(null);
        a.showAndWait();
    }

    public static class Mascota {
        int id, edad;
        String propietario, mascota, especie, diagnostico;

        Mascota(int i, String p, String m, String e, int ed, String d) {
            this.id = i;
            this.propietario = p;
            this.mascota = m;
            this.especie = e;
            this.edad = ed;
            this.diagnostico = d;
        }
    }
}

```

## 5. MYSQL LOCAL

```

package com.veterinaria;

import javafx.beans.property.SimpleIntegerProperty;
import javafx.beans.property.SimpleStringProperty;
import javafx.collections.FXCollections;
import javafx.collections.ObservableList;
import javafx.fxml.FXML;
import javafx.scene.control.Alert;
import javafx.scene.control.ComboBox;
import javafx.scene.control.TableColumn;
import javafx.scene.control.TableView;
import javafx.scene.control.TextField;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Statement;

public class VeterinariaController {

    @FXML
    private TextField txtPropietario, txtMascota, txtEdad, txtDiagnostico;
    @FXML
    private ComboBox<String> cmbEspecie;
    @FXML
    private TableView<Mascota> tablaMascotas;
    @FXML
    private TableColumn<Mascota, Integer> colId, colEdad;
    @FXML
    private TableColumn<Mascota, String> colPropietario, colMascota,
colEspecie, colDiagnostico;

    private final String URL =
"jdbc:mysql://localhost:3306/veterinaria?useSSL=false&serverTimezone=UTC";

```

```

private final String USUARIO = "root";
private final String CLAVE = "Jessi1995...";

private final ObservableList<Mascota> lista =
FXCollections.observableArrayList();

@FXML
public void initialize() {
    cmbEspecie.getItems().addAll("Perro", "Gato", "Ave", "Conejo",
    "Otro");

    colId.setCellValueFactory(d -> new
SimpleIntegerProperty(d.getValue().id).asObject());
    colPropietario.setCellValueFactory(d -> new
SimpleStringProperty(d.getValue().propietario));
    colMascota.setCellValueFactory(d -> new
SimpleStringProperty(d.getValue().mascota));
    colEspecie.setCellValueFactory(d -> new
SimpleStringProperty(d.getValue().especie));
    colEdad.setCellValueFactory(d -> new
SimpleIntegerProperty(d.getValue().edad).asObject());
    colDiagnostico.setCellValueFactory(d -> new
SimpleStringProperty(d.getValue().diagnostico));

    tablaMascotas.setItems(lista);
    cargarDatos();

    tablaMascotas.setOnMouseClicked(e -> {
        Mascota m =
tablaMascotas.getSelectionModel().getSelectedItem();
        if (m != null) {
            txtPropietario.setText(m.propietario);
            txtMascota.setText(m.mascota);
            cmbEspecie.setValue(m.especie);
            txtEdad.setText(String.valueOf(m.edad));
            txtDiagnostico.setText(m.diagnostico);
        }
    });
}

Connection conectar() throws SQLException {
    return DriverManager.getConnection(URL, USUARIO, CLAVE);
}

void cargarDatos() {
    lista.clear();
    String sql = "SELECT * FROM mascotas ORDER BY id DESC";

    try (Connection c = conectar();
        Statement s = c.createStatement();
        ResultSet r = s.executeQuery(sql)) {

        while (r.next()) {
            lista.add(new Mascota(
                r.getInt("id"),
                r.getString("propietario"),
                r.getString("mascota"),
                r.getString("especie"),
                r.getInt("edad"),

```

```

        r.getString("diagnostico")
    ));
    }
} catch (Exception e) {
    mensaje("Error", e.getMessage());
}
}

@FXML
void guardar() {
    if (!validar()) return;

    String sql = "INSERT INTO mascotas(propietario, mascota, especie,
edad, diagnostico) VALUES(?, ?, ?, ?, ?)";

    try (Connection c = conectar();
        PreparedStatement p = c.prepareStatement(sql)) {

        p.setString(1, txtPropietario.getText());
        p.setString(2, txtMascota.getText());
        p.setString(3, cmbEspecie.getValue());
        p.setInt(4, Integer.parseInt(txtEdad.getText()));
        p.setString(5, txtDiagnostico.getText());

        p.executeUpdate();
        limpiar();
        cargarDatos();
        mensaje("Correcto", "Mascota registrada correctamente.");
    } catch (Exception e) {
        mensaje("Error", e.getMessage());
    }
}

@FXML
void actualizar() {
    Mascota m = tablaMascotas.getSelectionModel().getSelectedItem();
    if (m == null) {
        mensaje("Aviso", "Seleccione un registro.");
        return;
    }

    if (!validar()) return;

    String sql = "UPDATE mascotas SET propietario=?, mascota=?,
especie=?, edad=?, diagnostico=? WHERE id=?";

    try (Connection c = conectar();
        PreparedStatement p = c.prepareStatement(sql)) {

        p.setString(1, txtPropietario.getText());
        p.setString(2, txtMascota.getText());
        p.setString(3, cmbEspecie.getValue());
        p.setInt(4, Integer.parseInt(txtEdad.getText()));
        p.setString(5, txtDiagnostico.getText());
        p.setInt(6, m.id);

        p.executeUpdate();
        limpiar();
        cargarDatos();
    } catch (Exception e) {
        mensaje("Error", e.getMessage());
    }
}

```

```

    }
}

@FXML
void eliminar() {
    Mascota m = tablaMascotas.getSelectionModel().getSelectedItem();
    if (m == null) {
        mensaje("Aviso", "Seleccione una mascota.");
        return;
    }

    String sql = "DELETE FROM mascotas WHERE id=?";

    try (Connection c = conectar();
        PreparedStatement p = c.prepareStatement(sql)) {

        p.setInt(1, m.id);
        p.executeUpdate();
        limpiar();
        cargarDatos();
    } catch (Exception e) {
        mensaje("Error", e.getMessage());
    }
}

@FXML
void nuevo() {
    limpiar();
    tablaMascotas.getSelectionModel().clearSelection();
}

boolean validar() {
    if (txtPropietario.getText().isBlank() ||
txtMascota.getText().isBlank() || cmbEspecie.getValue() == null) {
        mensaje("Validación", "Complete propietario, mascota y
especie.");
        return false;
    }

    try {
        if (Integer.parseInt(txtEdad.getText()) < 0) {
            throw new Exception();
        }
    } catch (Exception e) {
        mensaje("Validación", "La edad debe ser un número válido.");
        return false;
    }

    return true;
}

void limpiar() {
    txtPropietario.clear();
    txtMascota.clear();
    cmbEspecie.setValue(null);
    txtEdad.clear();
    txtDiagnostico.clear();
}

void mensaje(String t, String m) {
    Alert a = new Alert(Alert.AlertType.INFORMATION, m);

```

```

        a.setTitle(t);
        a.setHeaderText(null);
        a.showAndWait();
    }

    public static class Mascota {
        int id, edad;
        String propietario, mascota, especie, diagnostico;

        Mascota(int i, String p, String m, String e, int ed, String d) {
            this.id = i;
            this.propietario = p;
            this.mascota = m;
            this.especie = e;
            this.edad = ed;
            this.diagnostico = d;
        }
    }
}

```

## 6. RAILWAY

```

package com.veterinaria;

import javafx.beans.property.SimpleIntegerProperty;
import javafx.beans.property.SimpleStringProperty;
import javafx.collections.FXCollections;
import javafx.collections.ObservableList;
import javafx.fxml.FXML;
import javafx.scene.control.Alert;
import javafx.scene.control.ComboBox;
import javafx.scene.control.TableColumn;
import javafx.scene.control.TableView;
import javafx.scene.control.TextField;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Statement;

public class VeterinariaController {

    @FXML
    private TextField txtPropietario, txtMascota, txtEdad, txtDiagnostico;
    @FXML
    private ComboBox<String> cmbEspecie;
    @FXML
    private TableView<Mascota> tablaMascotas;
    @FXML
    private TableColumn<Mascota, Integer> colId, colEdad;
    @FXML
    private TableColumn<Mascota, String> colPropietario, colMascota,
colEspecie, colDiagnostico;

    private final String URL =

```

```

"jdbc:mysql://shuttle.proxy.rlwy.net:53581/railway";
    private final String USUARIO = "root";
    private final String CLAVE = "qWGtaSRQritGTgiLgzkeIgQoWNCeZgld";

    private final ObservableList<Mascota> lista =
FXCollections.observableArrayList();

    @FXML
    public void initialize() {
        cmbEspecie.getItems().addAll("Perro", "Gato", "Ave", "Conejo",
"Otro");

        colId.setCellValueFactory(d -> new
SimpleIntegerProperty(d.getValue().id).asObject());
        colPropietario.setCellValueFactory(d -> new
SimpleStringProperty(d.getValue().propietario));
        colMascota.setCellValueFactory(d -> new
SimpleStringProperty(d.getValue().mascota));
        colEspecie.setCellValueFactory(d -> new
SimpleStringProperty(d.getValue().especie));
        colEdad.setCellValueFactory(d -> new
SimpleIntegerProperty(d.getValue().edad).asObject());
        colDiagnostico.setCellValueFactory(d -> new
SimpleStringProperty(d.getValue().diagnostico));

        tablaMascotas.setItems(lista);
        cargarDatos();

        tablaMascotas.setOnMouseClicked(e -> {
            Mascota m =
tablaMascotas.getSelectionModel().getSelectedItem();
            if (m != null) {
                txtPropietario.setText(m.propietario);
                txtMascota.setText(m.mascota);
                cmbEspecie.setValue(m.especie);
                txtEdad.setText(String.valueOf(m.edad));
                txtDiagnostico.setText(m.diagnostico);
            }
        });
    }

    Connection conectar() throws SQLException {
        return DriverManager.getConnection(URL, USUARIO, CLAVE);
    }

    void cargarDatos() {
        lista.clear();
        String sql = "SELECT * FROM mascotas ORDER BY id DESC";

        try (Connection c = conectar();
            Statement s = c.createStatement();
            ResultSet r = s.executeQuery(sql)) {

            while (r.next()) {
                lista.add(new Mascota(
                    r.getInt("id"),
                    r.getString("propietario"),
                    r.getString("mascota"),
                    r.getString("especie"),

```

```

        r.getInt("edad"),
        r.getString("diagnostico")
    ));
    }
} catch (Exception e) {
    mensaje("Error", e.getMessage());
}
}

@FXML
void guardar() {
    if (!validar()) return;

    String sql = "INSERT INTO mascotas(propietario, mascota, especie,
edad, diagnostico) VALUES(?, ?, ?, ?, ?)";

    try (Connection c = conectar();
        PreparedStatement p = c.prepareStatement(sql)) {

        p.setString(1, txtPropietario.getText());
        p.setString(2, txtMascota.getText());
        p.setString(3, cmbEspecie.getValue());
        p.setInt(4, Integer.parseInt(txtEdad.getText()));
        p.setString(5, txtDiagnostico.getText());

        p.executeUpdate();
        limpiar();
        cargarDatos();
        mensaje("Correcto", "Mascota registrada correctamente.");
    } catch (Exception e) {
        mensaje("Error", e.getMessage());
    }
}

@FXML
void actualizar() {
    Mascota m = tablaMascotas.getSelectionModel().getSelectedItem();
    if (m == null) {
        mensaje("Aviso", "Seleccione un registro.");
        return;
    }

    if (!validar()) return;

    String sql = "UPDATE mascotas SET propietario=?, mascota=?,
especie=?, edad=?, diagnostico=? WHERE id=?";

    try (Connection c = conectar();
        PreparedStatement p = c.prepareStatement(sql)) {

        p.setString(1, txtPropietario.getText());
        p.setString(2, txtMascota.getText());
        p.setString(3, cmbEspecie.getValue());
        p.setInt(4, Integer.parseInt(txtEdad.getText()));
        p.setString(5, txtDiagnostico.getText());
        p.setInt(6, m.id);

        p.executeUpdate();
        limpiar();
        cargarDatos();
    } catch (Exception e) {

```



```

        mensaje("Error", e.getMessage());
    }
}

@FXML
void eliminar() {
    Mascota m = tablaMascotas.getSelectionModel().getSelectedItem();
    if (m == null) {
        mensaje("Aviso", "Seleccione una mascota.");
        return;
    }

    String sql = "DELETE FROM mascotas WHERE id=?";

    try (Connection c = conectar();
        PreparedStatement p = c.prepareStatement(sql)) {

        p.setInt(1, m.id);
        p.executeUpdate();
        limpiar();
        cargarDatos();
    } catch (Exception e) {
        mensaje("Error", e.getMessage());
    }
}

@FXML
void nuevo() {
    limpiar();
    tablaMascotas.getSelectionModel().clearSelection();
}

boolean validar() {
    if (txtPropietario.getText().isBlank() ||
txtMascota.getText().isBlank() || cmbEspecie.getValue() == null) {
        mensaje("Validación", "Complete propietario, mascota y
especie.");
        return false;
    }

    try {
        if (Integer.parseInt(txtEdad.getText()) < 0) {
            throw new Exception();
        }
    } catch (Exception e) {
        mensaje("Validación", "La edad debe ser un número válido.");
        return false;
    }

    return true;
}

void limpiar() {
    txtPropietario.clear();
    txtMascota.clear();
    cmbEspecie.setValue(null);
    txtEdad.clear();
    txtDiagnostico.clear();
}

void mensaje(String t, String m) {

```

```

        Alert a = new Alert(Alert.AlertType.INFORMATION, m);
        a.setTitle(t);
        a.setHeaderText(null);
        a.showAndWait();
    }

    public static class Mascota {
        int id, edad;
        String propietario, mascota, especie, diagnostico;

        Mascota(int i, String p, String m, String e, int ed, String d) {
            this.id = i;
            this.propietario = p;
            this.mascota = m;
            this.especie = e;
            this.edad = ed;
            this.diagnostico = d;
        }
    }
}

```

## 7. SUPABASE

```

8. package com.veterinaria;

import javafx.beans.property.SimpleIntegerProperty;
import javafx.beans.property.SimpleStringProperty;
import javafx.collections.FXCollections;
import javafx.collections.ObservableList;
import javafx.fxml.FXML;
import javafx.scene.control.Alert;
import javafx.scene.control.ComboBox;
import javafx.scene.control.TableColumn;
import javafx.scene.control.TableView;
import javafx.scene.control.TextField;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Statement;

public class VeterinariaController {

    @FXML
    private TextField txtPropietario, txtMascota, txtEdad,
    txtDiagnostico;
    @FXML
    private ComboBox<String> cmbEspecie;
    @FXML
    private TableView<Mascota> tablaMascotas;
    @FXML
    private TableColumn<Mascota, Integer> colId, colEdad;
    @FXML
    private TableColumn<Mascota, String> colPropietario,
    colMascota, colEspecie, colDiagnostico;
}

```

```

        private final String URL = "jdbc:postgresql://aws-0-us-east-1.pooler.supabase.com:5432/postgres";
        private final String USUARIO =
"postgres.ecndclkuuiyfkwcobpis";
        private final String CLAVE = "A.0106152176_.a";

        private final ObservableList<Mascota> lista =
FXCollections.observableArrayList();

        @FXML
        public void initialize() {
            cmbEspecie.getItems().addAll("Perro", "Gato", "Ave",
"Conejo", "Otro");

            colId.setCellValueFactory(d -> new
SimpleIntegerProperty(d.getValue().id).asObject());
            colPropietario.setCellValueFactory(d -> new
SimpleStringProperty(d.getValue().propietario));
            colMascota.setCellValueFactory(d -> new
SimpleStringProperty(d.getValue().mascota));
            colEspecie.setCellValueFactory(d -> new
SimpleStringProperty(d.getValue().especie));
            colEdad.setCellValueFactory(d -> new
SimpleIntegerProperty(d.getValue().edad).asObject());
            colDiagnostico.setCellValueFactory(d -> new
SimpleStringProperty(d.getValue().diagnostico));

            tablaMascotas.setItems(lista);
            cargarDatos();

            tablaMascotas.setOnMouseClicked(e -> {
                Mascota m =
tablaMascotas.getSelectionModel().getSelectedItem();
                if (m != null) {
                    txtPropietario.setText(m.propietario);
                    txtMascota.setText(m.mascota);
                    cmbEspecie.setValue(m.especie);
                    txtEdad.setText(String.valueOf(m.edad));
                    txtDiagnostico.setText(m.diagnostico);
                }
            });
        }

        Connection conectar() throws SQLException {
            return DriverManager.getConnection(URL, USUARIO,
CLAVE);
        }

        void cargarDatos() {
            lista.clear();
            String sql = "SELECT * FROM mascotas ORDER BY id DESC";

            try (Connection c = conectar();
                Statement s = c.createStatement();
                ResultSet r = s.executeQuery(sql)) {

                while (r.next()) {
                    lista.add(new Mascota(

```

```

        r.getInt("id"),
        r.getString("propietario"),
        r.getString("mascota"),
        r.getString("especie"),
        r.getInt("edad"),
        r.getString("diagnostico")
    ));
    }
} catch (Exception e) {
    mensaje("Error", e.getMessage());
}
}

@FXML
void guardar() {
    if (!validar()) return;

    String sql = "INSERT INTO mascotas(propietario,
mascota, especie, edad, diagnostico) VALUES(?, ?, ?, ?, ?)";

    try (Connection c = conectar();
        PreparedStatement p = c.prepareStatement(sql)) {

        p.setString(1, txtPropietario.getText());
        p.setString(2, txtMascota.getText());
        p.setString(3, cmbEspecie.getValue());
        p.setInt(4, Integer.parseInt(txtEdad.getText()));
        p.setString(5, txtDiagnostico.getText());

        p.executeUpdate();
        limpiar();
        cargarDatos();
        mensaje("Correcto", "Mascota registrada
correctamente.");
    } catch (Exception e) {
        mensaje("Error", e.getMessage());
    }
}

@FXML
void actualizar() {
    Mascota m =
tablaMascotas.getSelectionModel().getSelectedItem();
    if (m == null) {
        mensaje("Aviso", "Seleccione un registro.");
        return;
    }

    if (!validar()) return;

    String sql = "UPDATE mascotas SET propietario=?,
mascota=?, especie=?, edad=?, diagnostico=? WHERE id=?";

    try (Connection c = conectar();
        PreparedStatement p = c.prepareStatement(sql)) {

        p.setString(1, txtPropietario.getText());
        p.setString(2, txtMascota.getText());
        p.setString(3, cmbEspecie.getValue());
        p.setInt(4, Integer.parseInt(txtEdad.getText()));
        p.setString(5, txtDiagnostico.getText());
    }
}

```

```

        p.setInt(6, m.id);

        p.executeUpdate();
        limpiar();
        cargarDatos();
    } catch (Exception e) {
        mensaje("Error", e.getMessage());
    }
}

@FXML
void eliminar() {
    Mascota m =
tablaMascotas.getSelectionModel().getSelectedItem();
    if (m == null) {
        mensaje("Aviso", "Seleccione una mascota.");
        return;
    }

    String sql = "DELETE FROM mascotas WHERE id=?";

    try (Connection c = conectar();
        PreparedStatement p = c.prepareStatement(sql)) {

        p.setInt(1, m.id);
        p.executeUpdate();
        limpiar();
        cargarDatos();
    } catch (Exception e) {
        mensaje("Error", e.getMessage());
    }
}

@FXML
void nuevo() {
    limpiar();
    tablaMascotas.getSelectionModel().clearSelection();
}

boolean validar() {
    if (txtPropietario.getText().isBlank() ||
txtMascota.getText().isBlank() || cmbEspecie.getValue() ==
null) {
        mensaje("Validación", "Complete propietario,
mascota y especie.");
        return false;
    }

    try {
        if (Integer.parseInt(txtEdad.getText()) < 0) {
            throw new Exception();
        }
    } catch (Exception e) {
        mensaje("Validación", "La edad debe ser un número
válido.");
        return false;
    }

    return true;
}

```

```

        void limpiar() {
            txtPropietario.clear();
            txtMascota.clear();
            cmbEspecie.setValue(null);
            txtEdad.clear();
            txtDiagnostico.clear();
        }

        void mensaje(String t, String m) {
            Alert a = new Alert(Alert.AlertType.INFORMATION, m);
            a.setTitle(t);
            a.setHeaderText(null);
            a.showAndWait();
        }

        public static class Mascota {
            int id, edad;
            String propietario, mascota, especie, diagnostico;

            Mascota(int i, String p, String m, String e, int ed,
String d) {
                this.id = i;
                this.propietario = p;
                this.mascota = m;
                this.especie = e;
                this.edad = ed;
                this.diagnostico = d;
            }
        }
    }
}

```

## DEPENDENCIAS

```

<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd">
<modelVersion>4.0.0</modelVersion><groupId>com.veterinaria</groupId><artifa
ctId>VeterinariaApp</artifactId><version>1.0</version>
<properties><project.build.sourceEncoding>UTF-
8</project.build.sourceEncoding><maven.compiler.source>17</maven.compiler.s
ource><maven.compiler.target>17</maven.compiler.target></properties>
<dependencies>
<dependency><groupId>org.openjfx</groupId><artifactId>javafx-
controls</artifactId><version>21</version></dependency>
<dependency><groupId>org.openjfx</groupId><artifactId>javafx-
fxml</artifactId><version>21</version></dependency>
<dependency><groupId>com.mysql</groupId><artifactId>mysql-connector-
j</artifactId><version>8.4.0</version></dependency>

    <dependency>
        <groupId>com.mysql</groupId>
        <artifactId>mysql-connector-j</artifactId>
        <version>8.3.0</version>
    </dependency>

<dependency>

```

```
        <groupId>com.microsoft.sqlserver</groupId>
        <artifactId>mssql-jdbc</artifactId>
        <version>13.2.0.jre11</version>
    </dependency>

    <dependency>
        <groupId>org.mongodb</groupId>
        <artifactId>mongodb-driver-sync</artifactId>
        <version>5.6.0</version>
    </dependency>

    <!-- POSTGRESQL - SUPABASE -->
    <dependency>
        <groupId>org.postgresql</groupId>
        <artifactId>postgresql</artifactId>
        <version>42.7.7</version>
    </dependency>
</dependencies>
<build><plugins><plugin><groupId>org.openjfx</groupId><artifactId>javafx-
maven-
plugin</artifactId><version>0.0.8</version><configuration><mainClass>com.ve
terinaria.App</mainClass></configuration></plugin></plugins></build>
</project>
```